

Towards Practical Multi-Party Hash Chains using Arithmetization-Oriented Primitives

With Applications to Threshold Hash-Based Signatures

Alexandre Adomnicăi

DV Labs, USA

alexandre@dvlabs.tech

Abstract. Hash chains constitute a fundamental cryptographic primitive with widespread applications in authentication protocols and post-quantum cryptography. Despite their simplicity and quantum-resistant security properties, the deployment of hash chains in distributed settings through secure multi-party computation (MPC) has been demonstrated to be impractical when employing traditional hash functions (*i.e.*, SHA2/SHA3) due to their high number of non-linear gates which lead to heavy computational costs. In this work, we present a comprehensive evaluation of hash chain computations over MPC using arithmetization-oriented (AO) primitives, specifically focusing on the Poseidon2 family of hash functions. We systematically analyze the MPC-friendliness of various Poseidon2 instantiations across different prime fields and parameter choices to minimize both multiplicative depth and preprocessing requirements. We conduct extensive benchmarks using the MP-SPDZ framework across three state-of-the-art MPC protocols under varying network conditions and adversarial models. We further explore practical applications to threshold cryptography, presenting optimized implementations of threshold hash-based signatures that achieve signing times less than 1 second in a 3-party setting for practical parameter sets. Specifically, we demonstrate how structural parallelism in hash-based signatures can be exploited to batch independent hash chains within a single MPC execution, and introduce a time-memory trade-off that enables non-interactive online signature generation through systematic precomputation of all chain intermediates. Our work suggests the practical viability of moderate length AO-based hash chains for MPC applications.

Keywords: Hash chains, Multi-party computation, Hash-based signatures

1 Introduction

First introduced by Lamport in the context of one-time password authentication [Lam81], hash chains have become a core cryptographic primitive, widely used in protocols ranging from authentication mechanisms to post-quantum signature schemes. A hash chain consists of a sequence of values where each element is computed by applying a cryptographic hash function to the previous element, creating an irreversible chain of commitments. The security of hash chains relies fundamentally on the collision resistance and preimage resistance of the underlying hash function, making them particularly attractive for applications requiring long-term security guarantees. Notably, the security of hash chains is believed to remain intact even in the presence of cryptographically-relevant quantum computers, positioning them as one of the cornerstones of post-quantum signatures. However, the deployment of these primitives in modern distributed systems presents practical challenges that have not been adequately addressed in the literature. Today, many distributed

systems leverage threshold cryptography thanks to primitives whose algebraic structures allow native support for MPC deployments (*e.g.*, bilinear maps). For example, Ethereum’s consensus layer supports distributed validator solutions¹ via threshold BLS signatures, eliminating single points of failure in block proposal and attestation. Yet, the transition to post-quantum security will require replacing such schemes with quantum-safe security assumptions, including hash-based constructions, which lack the same algebraic properties. Hash-based signatures’ reliance on sequential structures such as hash chains and Merkle trees makes them far less amenable to MPC, raising new questions about how to securely and efficiently turn them into threshold schemes. Indeed, the practical deployment of hash chains over MPC has been shown to be inefficient because of the computational overhead when using cryptographic hash functions designed for efficient implementation in software or hardware. Traditional hash functions like SHA2 and SHA3, while efficient for direct computation, translate poorly to arithmetic circuits used in MPC, requiring thousands of multiplication gates and communication rounds. This overhead has resulted in the accepted belief that hash-based signatures over MPC are impractical, with previous work reporting computation times exceeding an hour for a single signature generation [CS19]. However, recent advances in arithmetization-oriented hash functions, exemplified by the Poseidon family of hash functions [GKR⁺21], have significantly improved the efficiency of hash computations within MPC protocols. These hash functions are specifically designed to minimize the number of multiplications in arithmetic circuits, making them well-suited for secure computation frameworks based on secret sharing, garbled circuits, and zero-knowledge proofs. Poseidon2 [GKS23], the latest iteration in this family, offers enhanced security margins while maintaining computational efficiency in arithmetic circuit representations. However, existing benchmarks and evaluations of AO hash functions have predominantly focused on their performance within zero-knowledge proof systems, where the primary concern is proof size and verification time. To the best of our knowledge, the applicability and efficiency of these hash functions in the context of secure multi-party computation, particularly for hash chain constructions underlying threshold hash-based signatures and distributed password authentication systems, remain to be comprehensively evaluated. This gap is significant, as the performance characteristics and optimization strategies for MPC protocols differ substantially from those of zero-knowledge systems, involving different trade-offs between communication/round complexity and computational overhead.

Our contributions. This work evaluates the practical relevance of hash chains over secure multi-party computation by reporting detailed performance benchmarks across different MPC protocols and optimization strategies. More precisely, we consider the Poseidon2 hash function whose security has received extensive cryptanalysis since its original publication². While our benchmarks suggest that AO hash chains exceeding a length of ≈ 100 are not efficient enough for distributed deployments, chains an order of magnitude shorter execute in under 1 second on a LAN and within a few seconds on a 20ms round-trip time network. Because this aligns well with hash-based signatures, which generally rely on chains of this size, we further explore applications to threshold hash-based signatures and present implementation techniques that significantly enhance their efficiency in an MPC setting. The first technique leverages structural parallelization, exploiting the independence of hash chains within one-time signature constructions to amortize communication costs. The second introduces a time-memory trade-off, enabling non-interactive online signature generation through systematic precomputation. When combined, these techniques yield experimental benchmark results demonstrating that threshold hash-based signatures over MPC can be computed in under 1 second per signature, in a 3-party setting, on a 20ms

¹<https://github.com/ethereum/distributed-validator-specs>

²<https://www.poseidon-initiative.info/>

round-trip time network. The remainder of this paper is organized as follows. Section 2 provides the necessary background on hash chains, secure multi-party computation, and AO hash functions. In Section 3, we carefully consider various prime fields to identify the most MPC-friendly instantiations. Among the candidates considered, the KoalaBear prime field stands out by delivering the best MPC metrics overall. Section 4 presents comprehensive experimental results across different MPC protocols and parameter settings. Section 5 explores applications to threshold hash-based signatures, with a specific focus on the generalized XMSS variant which is currently considered for integration into Ethereum’s consensus layer as part of its post-quantum migration [DKKW25a]. Finally, Section 6 summarizes our results and outlines directions for future research.

2 Background

2.1 Hash Chains

2.1.1 Definition

A hash chain is a sequence of values constructed by repeatedly applying a cryptographic hash function. Formally, given a cryptographic hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ and an initial seed value $s_0 \in \{0, 1\}^*$, a hash chain of length ℓ is defined as:

$$s_i = H(s_{i-1}) \quad \text{for } 1 \leq i \leq \ell.$$

The resulting sequence $(s_0, s_1, \dots, s_\ell)$ forms the hash chain, where s_0 being the seed and s_ℓ the final value, which is usually made public. Each element s_i serves as a cryptographic commitment to all preceding elements in the chain, creating an irreversible sequence of linked values.

2.1.2 Security Properties

The security of hash chains relies on the cryptographic properties of the underlying hash function H .

Preimage Resistance. Given an intermediate value s_i in the chain, it should be computationally infeasible to find the preceding value s_{i-1} such that $H(s_{i-1}) = s_i$. This property ensures that knowledge of a later element in the chain does not reveal information about earlier elements.

Second Preimage Resistance. Given an element s_i in the chain, it should be computationally infeasible to find a different value $s'_i \neq s_i$ such that $H(s'_i) = H(s_i) = s_{i+1}$. This prevents adversaries from creating alternative chain prefixes.

Collision Resistance. It should be computationally infeasible to find two distinct values $x, x' \in \{0, 1\}^*$ such that $H(x) = H(x')$. These properties ensure that hash chains provide forward security: knowledge of elements $(s_i, \dots, s_\ell)$ does not compromise the security of earlier elements $(s_0, \dots, s_{i-1})$.

2.1.3 Mainstream Applications

One-time password authentication schemes. The idea of a one-time password authentication scheme was first considered by Lamport [Lam81] where he proposed a concrete instantiation using hash chains, which has been subsequently specified under the name S/Key in RFC 1760 [Hal95]. The setup phase of S/Key consists of the client choosing a

secret seed x and sending the computed value $y_0 = H^k(x)$ (where H is some cryptographic hash function, k is some integer, and H^k denotes k successive iterations of H) to the server, which the server then stores. Subsequently, to authenticate for the i -th time, the client must present the server with $y_i = H^{k-i}(x)$, which the server can verify by computing $H(y_i)$ and comparing it to the stored value y_{i-1} . If the authentication is successful, the server updates its stored value to y_i . This construction suffers from two main weaknesses: passwords remain valid indefinitely (at least, until the next authentication round takes place), and the repeated iteration of the same one-way function makes it weaker than a single hash function call. Later, the T/Key [KMB17] and U/Key [LM24] constructions were proposed as time-based extensions of S/Key, incorporating temporal constraints to address the issue that S/Key passwords remain valid indefinitely and further enhancing security. In its original publication, the T/Key scheme recommends chains of 2 000 hash operations to support a two-year credential validity period assuming 30-second authentication windows per one-time password.

Hash-based signatures. Winternitz signatures constitute a fundamental construction in hash-based cryptography, leveraging the one-way property of cryptographic hash functions to create digital signatures resistant to quantum attacks. The scheme operates by dividing the message digest into blocks of w bits (the Winternitz parameter) and constructing a separate hash chain for each block, where the private key consists of random starting values and the public key comprises the chains' endpoints. During signing, the signer reveals specific intermediate values within these chains determined by the message content, allowing verification without compromising the private key. In standardized schemes like XMSS [HBG⁺18] and LMS [MCF19], typical Winternitz parameter values range from $w = 1$ to $w = 16$, resulting in hash chains of lengths up to 65 536, with practical implementations commonly employing chain lengths of 16 or 256 elements as a compromise between signature size and computation cost. Indeed, these parameters directly impact the security-efficiency tradeoff, with larger w values reducing signature size at the expense of increased computational overhead, a critical consideration for resource-constrained environments.

2.2 MPC-Friendly Hash Functions

2.2.1 Secret Sharing Based MPC

MPC enables multiple parties to jointly and securely compute a function over their private inputs while ensuring that no party learns anything beyond what can be inferred from their own input and the result. In this work, we focus on linear secret sharing based MPC where each party's private input is distributed among all parties using a (t, n) -threshold secret sharing scheme over a prime field. We use the bracket notation $[\cdot]$ to denote shared secrets. Thanks to the linear property of the underlying secret sharing scheme, the parties can locally (*i.e.*, non-interactively) add their shares of two secrets to obtain shares of the sum. Still, nonlinear operations such as multiplications with shared operands require communication between the parties. A widely adopted approach to reduce the communication complexity of multiplications in MPC is the use of Beaver triples [Bea92] that introduced the now-standard offline/online paradigm to MPC protocols. The key insight is to separate computation into two phases: an offline phase that preprocesses data-independent multiplication triples $([a], [b], [a \cdot b])$, and a subsequent online phase that consumes these triples to perform actual multiplications in a single communication round. When it comes to exponentiations, while it is always possible to compute multiplications in chain, advanced methods have been proposed to further reduce the communication complexity. For cube evaluations, Grassi, Rechberger, Rotaru, Scholl, and Smart proposed a method [GRR⁺16] which makes the offline phase more computation intensive but where

the online phase only consumes two triples and a single communication rounds. Actually, this method can be generalized to any exponent d as follows. Given preprocessed tuples $([r], [r^2], \dots, [r^d])$ one can always compute $[x^d]$ in a single online round which consists in opening $y = [x] - [r]$ so that $[x^d] = \sum_{i=0}^d \binom{d}{i} y^i [r^{d-i}]$ requires no additional interaction. Whereas independent multiplications can be batched together to lower the number of communication rounds, the *multiplicative depth* defines the minimum number of online communication rounds required. Overall, the multiplicative depth as well as the amount of preprocessed data are key metrics to assess the MPC-friendliness of a primitive.

2.2.2 Arithmetization-Oriented Primitives

Traditional hash functions such as SHA2 and SHA3 operate over binary fields to enable efficient implementations in both hardware and software on a wide range of platforms. However, they lead to poor performance when employed within advanced cryptographic protocols such as MPC. This is mainly due to the fact that traditional schemes are designed to minimize their overall gate count without minimizing specifically nonlinear gates³ which require communication between parties in an MPC setting, unlike linear gates that can be computed locally. The overload induced by these communication rounds is such that it can constitute the bottleneck in MPC protocols. In response, new primitives with design constraints finely tuned for advanced cryptographic protocols have emerged, known as *arithmetization-oriented* primitives. They usually operate over prime fields, making them natively compatible with linear secret sharing schemes. Still, all AO hash functions are not necessarily optimized for MPC, as some of them are primarily ZKP-oriented and leverage proof-system features that have no analogue in MPC. In the MPC setting, efficiency is dictated almost entirely by the number of multiplications and the multiplicative depth of the circuit, since each multiplication requires costly interaction among the parties. By contrast, many recent ZKP-specific designs achieve efficiency by relying on advanced proving-system machinery, such as lookup arguments or custom gates, which render complex nonlinear layers essentially “free” inside a proof but remain expensive in a pure arithmetic circuit. As a result, hash functions such as Monolith [GKL⁺24] are highly competitive in zero-knowledge systems but translate poorly into MPC protocols. In contrast, families like Poseidon/Poseidon2 [GKR⁺21, GKS23], Rescue [AABS⁺20], or Hydra [GØSW23] are explicitly designed to minimize multiplicative complexity over prime fields, making them more suitable for deployment in MPC frameworks. Because the hash-based signature framework we aim to thresholdize supports instantiations with Poseidon2, our analysis will focus on this specific hash function, introduced in detail below.

2.2.3 The Poseidon2 Family of Hash Functions

The Poseidon2^π permutation. Poseidon2 is built upon the Poseidon2^π permutation operating over $\mathbb{F}_p^t$ with $p > 2^{30}$ prime and $t \in \{2, 3, 4, 8, 12, 16, 20, 24\}$. Poseidon2^π, optimizes the linear layers of its predecessor Poseidon to improve efficiency and mitigate the attack proposed in [BBLP22]. Poseidon2^π is based on the HADES design strategy [GLR⁺20] which makes a distinction between external and internal rounds. Internal rounds (also called partial rounds) apply the nonlinear layer to only a part of the state, usually a single element, whereas external rounds (also called full rounds) process all elements in the same way. More precisely, Poseidon2^π processes an internal state $x = (x_0, \dots, x_{t-1}) \in \mathbb{F}_p^t$ as follows:

$$\text{Poseidon2}^\pi(x) = \mathcal{E}_{R_F-1} \circ \dots \circ \mathcal{E}_{R_F/2} \circ \mathcal{I}_{R_P-1} \circ \dots \circ \mathcal{I}_0 \circ \mathcal{E}_{R_F/2-1} \circ \dots \circ \mathcal{E}_0(M_{\mathcal{E}} \cdot x)$$

³They are actually symmetric designs that aim at minimizing the number of nonlinear gates for efficient software masked implementations against side-channel attacks, see for instance [GLSV14].

where $\mathcal{E}$ and $\mathcal{I}$ refer to external and internal round functions iterated for R_F and R_P rounds, respectively. Note that a linear layer is applied before running the first external round, which differs from the original Poseidon $^\pi$ design. The external/full round function is defined by:

$$\mathcal{E}(x) = M_{\mathcal{E}} \cdot \left((x_0 + c_0^{(i)})^d, \dots, (x_{t-1} + c_{t-1}^{(i)})^d \right)$$

where $d \geq 3$ is the smallest integer such that $\gcd(d, p-1) = 1$, $M_{\mathcal{E}}$ is a $t \times t$ maximum distance separable (MDS) matrix and $c_j^{(i)}$ is the j -th round constant for the i -th external round. The internal/partial round function is defined by:

$$\mathcal{I}(x) = M_{\mathcal{I}} \cdot \left((x_0 + \hat{c}_0^{(i)})^d, x_1, \dots, x_{t-1} \right)$$

where $d \geq 3$ as before, $M_{\mathcal{I}}$ is a $t \times t$ MDS matrix and $\hat{c}_0^{(i)}$ is the round constant for the i -th internal round.

Modes of operation. While Poseidon is specified as a sponge construction, Poseidon2 also supports compression functions which are recommended for 2-to-1 settings like Merkle hashing. A compression function reduces a larger input to a fixed-size output through one or more applications of the underlying permutation. In its original publication, the single call compression function specified for Poseidon2 is defined as follows

$$\begin{aligned} \text{Poseidon2Compress}_{p,t,u}(\mathbf{x}) : \mathbb{F}_p^t &\longrightarrow \mathbb{F}_p^u \\ \mathbf{x} &\longmapsto \text{Truncate}_u(\text{Poseidon2}^\pi(\mathbf{x}) + \mathbf{x}) \end{aligned}$$

where $\text{Truncate}_u(\mathbf{x})$ extracts the first u elements of $\mathbf{x} \in \mathbb{F}_p^t$ and $+$ is the elementwise addition in $\mathbb{F}_p^t$. In contrast, the sponge construction absorbs the input data into the internal state of the permutation in successive steps, followed by a squeezing phase that extracts the digest. It accommodates for both arbitrarily sized inputs and outputs at the expense of some computational overhead.

3 MPC-Friendly Instantiations of Poseidon2

3.1 Related Work on Poseidon in MPC

Poseidon was designed under the HADES strategy which aims to minimize arithmetic-circuit costs. Although it was designed for advanced cryptographic protocols such as MPC/FHE/ZK, empirical work has mainly targeted proof systems rather than MPC, as highlighted by the original paper reporting results primarily in zkSNARK/zkSTARK settings rather than MPC applications. As a result, MPC benchmarking of Poseidon remains comparatively limited. Aly, Ashur, Ben-Sasson, Dhooghe and Szeponiec [AABS⁺20] provide one of the few systematic evaluations, reporting multiplication counts and online-round latency for several AO primitives, including Poseidon. More recently, Yıldız and Maller [YM24] compared Poseidon and Hydra across both ZK and MPC use cases, finding Hydra generally more efficient for MPC workloads. However, we believe that neither of these studies reflects what can be optimally achieved with Poseidon2. First, the performance of instances over small prime fields (*e.g.*, 31-bit) have not yet been explored. Second, unlike its predecessor, Poseidon2 introduces a dedicated compression mode which reduces state size compared to instantiations with the sponge construction, decreasing the total number of multiplications. Hereafter we investigate the most MPC-friendly instantiations with a focus on the compression mode as it fits perfectly with hash chains.

3.2 Concrete Poseidon2 $^\pi$ Instantiations

We now describe our MPC-friendly instantiations of Poseidon2 $^\pi$ in compression mode, emphasizing parameter choices and efficiency metrics relevant to hash-chain computations. From an MPC perspective, the dominant online cost is the multiplicative depth and the dominant offline cost is the number of preprocessed field elements (*e.g.*, Beaver triples) consumed by those multiplications. For Poseidon2 $^\pi$, all the nonlinearity resides in the S-box $x \mapsto x^d$; the linear layers (mixing by an MDS matrix and the round constants) are free in standard arithmetic-MPC models because they involve only additions and multiplications by public constants. While common instinct is to minimize the exponent and adopt $d = 3$, which indeed minimizes both the number of multiplications per S-box and the per-round multiplicative depth, the permutation’s round count is equally critical as depth accumulates across rounds. Writing $\delta(d)$ for the minimal depth of x^d , and R_F, R_P for the numbers of full and partial rounds, the permutation-level depth is defined by

$$\text{depth}(\pi) = (R_F + R_P) \cdot \delta(d). \quad (1)$$

This makes clear why optimizing only d is insufficient: an apparently “cheap” S-box with a large R_P can end up deeper than a slightly more expensive S-box paired with fewer rounds.

Rounds as a function of (n, t, d) . The number of rounds in Poseidon2 is derived from the parameter triple (n, t, d) , where n is the bit-length of the prime field $\mathbb{F}_p$, t is the state width, and d is the S-box exponent. Security analyses fix R_F and determine R_P as a function of (n, t, d) . In particular, R_F is kept constant at eight full rounds, while R_P decreases as either the state becomes wider or the exponent grows. Intuitively, more diffusion per round (larger t) or a higher algebraic degree per S-box (larger d) requires fewer partial rounds to reach the same security margins. These dependencies have different consequences regarding MPC-friendly instantiations. A smaller field typically forces a larger state to maintain capacity, which increases the number of S-boxes in each full round. This raises the count of multiplications, but does not increase multiplicative depth, since all those per-round S-boxes are parallelizable. At the same time, wider states usually allow a smaller R_P , which reduces both depth (fewer rounds) and the total number of S-box evaluations across the permutation. Thus, when moving across fields one observes two opposing effects on the offline footprint: wider t increases the number of S-boxes in full rounds, while the accompanying drop in R_P (and, hence, in the number of partial-round S-boxes) decreases it. The same tension arises when choosing the exponent. Larger d increases the per-S-box cost (both the number of secret–secret multiplications and $\delta(d)$), but it reduces R_P for a fixed (n, t) . Conversely, $d = 3$ gives the cheapest S-box, yet often requires more partial rounds to meet security. Putting these elements together, the preprocessing footprint is well approximated by

$$\text{prec}(\pi) = \mu(d) \cdot (R_F \cdot t + R_P), \quad (2)$$

where $\mu(d)$ is the number of multiplications needed for one S-box under the chosen implementation of x^d . This expression captures two key facts for MPC: the footprint scales with the number of S-boxes and partial rounds contribute only one S-box each, so reducing R_P can have an outsized effect on preprocessing, especially when t is small.

Implementing power maps in MPC. Motivated by these considerations, we evaluate several Poseidon2 $^\pi$ instances across prime fields of 31, 64, and 256 bits and for $t \in \{16, 8, 2\}$, respectively, to focus on 2-to-1 compressions. To cover multiple cases we consider exponents $d \in \{3, 5, 7\}$ and report the resulting multiplicative depth and the number of preprocessed field elements required in the offline phase. We consider the following two implementation strategies for x^d :

Table 1: MPC metrics of Poseidon2 $^\pi$ instances for 31, 64 and 256-bit prime fields, focusing on 2-to-1 compressions. For $d > 3$, both single-round exponentiations and square/cube decompositions are considered.

Parameters (n, t, d)	Sbox impl.		MPC metrics	
	R_F	R_P	depth(π)	prec(π)
(31, 16, 3)	8	20	x^3	28 296
(31, 16, 5)	8	14	x^5	22 568
			$(x^2)^2 \cdot x$	66 426
(31, 16, 7)	8	13	x^7	21 846
			$(x^2)^3 \cdot x$	63 564
(64, 8, 3)	8	41	x^3	49 210
(64, 8, 5)	8	27	x^5	35 364
			$(x^2)^2 \cdot x$	105 273
(64, 8, 7)	8	22	x^7	30 516
			$(x^2)^3 \cdot x$	90 344
(256, 2, 3)	8	83	x^3	91 198
(256, 2, 5)	8	56	x^5	64 288
			$(x^2)^2 \cdot x$	192 216
(256, 2, 7)	8	46	x^7	54 372
			$(x^2)^3 \cdot x$	162 248

1. **One-round exponentiation.** A generalized one-round x^d evaluation with a single interactive multiplication layer. Consequently, the multiplicative depth equals $R_F + R_P$, independent of d . The offline cost scales with the number of S-boxes and the multiplication count per S-box for that exponent (*e.g.*, x^3 uses two triples per S-box). This is the latency-optimal choice.
2. **Square/cube decompositions.** Exponents such as $d = 5, 7$ can be decomposed using cubes and squares as $x^5 = ((x^2)^2) \cdot x$ and $x^7 = ((x^3)^2) \cdot x$. Both circuits require three interactive layers for each S-box, leading to a multiplicative depth equals to $3(R_F + R_P)$. The advantage is a lower triple budget per S-box, which reduces the offline preprocessing load.

Results interpretation. The MPC metrics results, summarized in Table 1, suggest that an instance with parameters (31, 16, 3) achieves the best tradeoff in terms of multiplicative depth and preprocessing requirements. Moreover, relying on a smaller prime field obviously reduce bandwidth consumption and storage for preprocessed elements. As a concrete example, the 198 required field elements for (256, 2, 3) weight 6.19KiB overall whereas the 296 field elements for (31, 16, 3) only weight 1.12KiB. In the rest of this paper, we focus on this specific instantiation using the KoalaBear prime defined by $p = 2^{31} - 2^{24} + 1$.

4 Experimental Benchmarks

4.1 Benchmarking Setup

The MP-SPDZ framework. Benchmarking MPC protocols is challenging not only due to their technical complexity but also because their efficiency depends on various factors such as the security model, the number of participants and network conditions. In recent years, various frameworks have been developed to push the boundaries of practical MPC by making it more accessible. Among them, MP-SPDZ [Kel20] is very popular and actively maintained in an open source manner⁴. Users can write programs in high-level Python code, which is then extensively optimized and compiled into bytecode for a virtual machine implemented in C++. The framework implements a wide variety of protocols for several different security models and performance characteristics, for both protocols based on secret sharing and garbled circuits. For our experiments, we utilized MP-SPDZ version v0.4.1.

Setup. We evaluate hash chain lengths $\ell \in \{8, 128, 512\}$ ranging from lightweight to more demanding computations. Benchmarks were run on an ASUS UX3405 laptop equipped with a Core Ultra 7 155H CPU. We considered a three-party setting over a network with 1Gbps bandwidth, and distinct single-trip delay latencies to capture different realistic deployment settings⁵:

- 1ms one-way delay to reflect participants within the same data center or cloud region;
- 10ms one-way delay to reflect geographically distributed parties within the same continent (*e.g.*, participants across Europe);
- 100ms one-way delay to reflect intercontinental communication (*e.g.*, Europe $\leftrightarrow$ Asia).

While our benchmarks were run on single machine, real distributed environments were simulated by using the linux `tc` command from the network emulation package `netem`⁶ to modify bandwidth and latency. We stress that the resulting round-trip time (RTT) is twice the single-trip latency as it defines a two-way delay. The code used for benchmarking is publicly available at https://github.com/aadomn/mpspd_z_poseidon2.

4.2 MPC Protocols

We summarize below the three protocols we considered for our benchmarks.

MASCOT. The MASCOT protocol [KOS16] realizes actively secure MPC for arithmetic circuits over prime fields in the dishonest-majority setting. The adversary is assumed to be static, meaning that the set of corrupted parties is determined before the onset of the protocol and cannot change throughout its execution. It employs a two-phase structure: an offline phase that leverages oblivious transfer (OT) extensions to generate authenticated multiplication triples, and an online phase that executes efficient linear-time computations using additive secret shares and global message authentication codes (MACs). Its innovation lies in deriving arithmetic correlations directly from bitwise OT, eliminating reliance on expensive public-key or cut-and-choose methods. Its security is computational, relying on the hardness assumptions underlying oblivious transfer (OT) and symmetric-key primitives. Additionally, it also relies on a statistical security parameter κ as triple-consistency and MAC checks detect active cheating with probability $2^{-\kappa}$.

⁴<https://github.com/data61/MP-SPDZ>

⁵<https://wondernetwork.com/pings>

⁶<https://wiki.linuxfoundation.org/networking/netem>

ATLAS. The ATLAS protocol [GLO⁺21] realizes unconditionally secure MPC for arithmetic circuits in the honest-majority setting, offering both actively secure and passively secure variants. Its main innovation lies in exploiting t -wise independence of correlated randomness: by generating only t pairs of random double sharings and expanding them to n pairs with t -wise independence, ATLAS reduces the amortized cost of multiplication gates while maintaining security against corrupted parties who handle at most t multiplications. The protocol achieves a communication complexity of 4 field elements per party per multiplication gate in the semi-honest setting (compared to 6 in the classic DN [DN07] protocol), and additionally offers a round-optimized variant that evaluates two circuit layers in parallel using a novel interplay between Beaver triples and the DN multiplication protocol, reducing rounds by half with 4.5 elements per party per gate. Since MP-SPDZ does not currently support ATLAS’ active security variant⁷ nor its round-reduced method⁸, our benchmarks rely on the variant that provides passive security using the t -wise method.

Malicious Shamir. The Malicious Shamir protocol achieves active security for arithmetic circuits on Shamir secret sharing in the honest-majority setting. It is obtained from a specific instantiation of a generic framework [LN17] that allows to turn a large class of MPC protocols secure in the semi-honest setting into protocols secure against malicious adversaries. MP-SPDZ implements the variant which relies on the GRR multiplication protocol [GRR98] which leads to reduced online round complexity at the expense of increased communication complexity. In contrast to dishonest-majority protocols such as MASCOT, Malicious Shamir provides information-theoretic security but still relies on a statistical parameter κ to detect deviations with probability $2^{-\kappa}$ when reconstructing or authenticating the shares.

For the actively secure protocols MASCOT and Malicious Shamir, we reduced the statistical security parameter from the MP-SPDZ default of 40 bits to 31 bits to ensure compatibility with the finite field defined by the KoalaBear prime. However, because the adequate statistical security parameter highly depends on the application (*e.g.*, time authentication windows duration and number of one-time signatures for one-time password authentication and stateful hash-based signatures, respectively), a 31-bit security level may be insufficient in some cases. Therefore, to enable a 40-bit statistical security level, we also consider the MAMA protocol provided by MP-SPDZ, a MASCOT variant with several MACs to increase the security parameter, which is especially relevant when relying on small prime fields as in our case. While MP-SPDZ defaults the number of MACs to three for MAMA, we set it to two as it was sufficient to achieve 40 bits statistical security when working with the KoalaBear prime. Regarding ATLAS and Malicious Shamir protocols, we stress that while they both theoretically provide information-theoretic setting, MP-SPDZ actually implements them in the computational setting since it relies on plenty of components with computational security (*e.g.*, AES-based PRNG).

4.3 Results Interpretation

Our experimental results, reported in Table 2, reveal significant performance variations across different security models and network conditions. In the honest majority setting with minimal network latency (1ms), protocols demonstrate practical efficiency, computing hash chains of length 8 in under one second. Since execution times scale linearly with chain length, one can easily derive estimates for other ℓ parameters. This performance baseline establishes the practical feasibility of short multi-party hash chain under idealized network conditions. However, when considering more distributed deployment scenarios,

⁷<https://github.com/data61/MP-SPDZ/issues/1578>

⁸<https://github.com/data61/MP-SPDZ/issues/676>

Table 2: Benchmarks of hash chains over MPC using the MP-SPDZ framework, in a three-party setting, where ℓ refers to the hash chain length. Timing executions are reported for different RTT delays (1ms, 10ms and 100ms) to reflect various network conditions.

ℓ	Protocol	Prec.	Offline phase				Online phase				
			Time (s)			Data (MB)	Comm. rounds	Time (s)			Data (MB)
			$d = 1$	$d = 10$	$d = 100$			$d = 1$	$d = 10$	$d = 100$	
8	MAMA	2 368	0.37	1.82	18	38	1 485	1.32	13	126	0.09
	MASCOT	2 368	0.23	1.30	13	17	1 485	1.32	13	126	0.08
	ATLAS	1 184	0.01	0.09	0.81	0.08	454	0.48	4.65	45	0.06
	Mal. Shamir	2 368	0.02	0.17	1.61	0.17	226	0.25	2.32	23	0.06
128	MAMA	37 888	4.97	22	238	576	23 563	21	201	2 000	1.39
	MASCOT	37 888	2.29	9.72	90	269	23 563	21	201	1 999	1.33
	ATLAS	18 944	0.03	0.15	1.41	0.28	7 176	7.66	73	719	0.51
	Mal. Shamir	37 888	0.06	0.39	3.64	2.21	3 584	3.93	37	360	0.91
512	MAMA	151 552	20	87	943	2 305	94 220	82	805	7 996	5.55
	MASCOT	151 552	8.85	38	348	1 076	94 220	83	805	7 996	5.33
	ATLAS	75 776	0.07	0.49	4.46	1.14	28 692	30	291	2 876	2.03
	Mal. Shamir	151 552	0.23	1.37	12	8.83	14 336	16	147	1 439	3.64

the performance characteristics shift considerably. Under a 10ms latency configuration, representative of high-speed connections within the same continent, even small hash chains of length 8 require several seconds to compute. We remark that for $d \in \{10, 100\}$, the online execution time can be reasonably approximated as $\text{network_latency} \times \#com_rounds$, as network delay dominates computational overhead. This shows how sensitive these protocols are to network conditions and highlights the importance of geographic distribution as a key factor in assessing deployment practicality. In particular, this becomes even more challenging when more powerful adversarial models are considered. For MASCOT and its MAC-duplicated variant MAMA, which provide active security in the dishonest majority setting, even the smallest hash chains demand several seconds of computation time despite operating under the favorable 1ms latency condition. Nevertheless, these findings remain encouraging for certain use cases. Indeed, hash-based signature schemes typically employ relatively short hash chains in practical deployments as mentioned in Section 2. This practical constraint motivates further investigation into threshold variants of hash-based signatures that leverage secret sharing schemes along with MPC protocols.

5 Applications to Threshold Hash-Based Signatures

5.1 Hash-Based Signatures

5.1.1 Hash-Based Signatures

Hash-based signature (HBS) schemes [Rab78, Lam79, Mer79] are one of the main families of post-quantum signatures, with security grounded in the pre-image and collision resistance of cryptographic hash functions. They rely on one-time signatures (OTS) as building blocks, which are then combined into more sophisticated tree-based structures to enable multiple signatures from a single key pair.

One-time signatures via hash chains. An OTS scheme is parameterized by:

- The number of hash chains v .
- The length of hash chains w .
- An n -bit hash function $H: \{0, 1\}^* \rightarrow \{0, 1\}^n$.
- An encoding function $E: \{0, 1\}^* \rightarrow \{1, \dots, w\}^v$.

The secret key consists of v n -bit components $sk_1, \dots, sk_v$ from which public key components $pk_i = H^{w-1}(sk_i)$ are derived by applying hash chains of length $w - 1$. The signature also consists of v components $\sigma_i = H^{x_i-1}(sk_i)$ where the chain positions x_i are obtained from the encoded message $E(m)$. To verify a signature, one simply checks that $H^{w-x_i}(\sigma_i) = pk_i$ for all components. One may note that the encoding function is an important building block as it defines the number of hash chains and hence directly impacts the key and signature sizes. Moreover, it is also responsible for the existential unforgeability of the scheme by either incorporating a checksum or ensuring all encodings lead to the same sum $T = \sum_i x_i$. The latter method is called constant (or target) sum Winternitz and was first introduced by Bos and Chaum [BC93] and Vaudenay [Vau93].

Merkle trees. To build a many-time signature scheme from OTS, one can combine multiple key pairs with a binary tree where each node is the hash of its children, commonly referred to as Merkle tree. For a height parameter h , such a tree is built from 2^h leaves $l_1, \dots, l_{2^h}$, each being the hash of an OTS public key (*i.e.*, $l_i = H(pk_{i_1}, \dots, pk_{i_v})$). The root constitutes the many-time public key and commits to all OTS public keys. Note that to reduce memory requirements in practice, OTS secret keys are usually generated using a pseudorandom function (PRF). To sign the i th message, the signer uses the i th OTS secret key and includes the Merkle path of the corresponding public key in the signature. To verify the signature, the verifier computes the public key from OTS signature and then, thanks to the Merkle path, verifies that its digest is indeed the leaf at position i . This introduces the concept *statefulness*: because security depends on the unique usage of each OTS key pair, it is crucial to keep track of which keys have already been used.

State management. Examples of stateful HBS includes XMSS [BDH11] and LMS [LM95], both standardized by IETF [HBG⁺18, MCF19] and NIST [CAD⁺20]. Keeping track of the state must be done very carefully, as a missed update compromises the cryptosystem's security. Consequently, practical deployment of stateful HBS poses challenges, prompting NIST to recommend that key and signature generation be performed exclusively in hardware security modules [CAD⁺20]. By leveraging hypertree structures that allow pseudorandom key derivation rather than sequential key management, it is possible to build *stateless* HBS schemes such as SPHINCS+ [BHK⁺19]. While stateless approaches solve the state management problem, they come at the cost of more hash computations and larger signature sizes. Conversely, proof-of-stake blockchains present an ideal use case for stateful HBS as the one-signature-per-epoch model naturally accommodates state management. This fit has prompted such applications to investigate stateful HBS as a post-quantum upgrade, as illustrated by the scheme summarized below.

5.1.2 LeanSig

The LeanSig stateful HBS framework was introduced by Drake, Khovratovich, Kudinov, and Wagner [DKKW25a] as part of the Ethereum consensus layer redesign⁹ which aims to provide post-quantum security. This framework generalizes XMSS by introducing the notion of *incomparable encoding schemes* to abstract the encoding layer (*i.e.*, the mapping from m to $x_1, \dots, x_n$). The choice of the underlying hash function and incomparable encoding scheme leads to several variants of XMSS. Notably, all instantiations enjoy the same security proof which does not model hash functions as random oracles and relies on standard model properties instead, such as preimage/collision resistance, providing concrete security targets. In a subsequent work Khovratovich, Kudinov, and Wagner [KKW25] have further explored incomparable encoding schemes with the goal of minimizing verification costs given a signature size and a security level. On top of providing a novel lower

⁹<https://leanroadmap.org/>

bound, they presented new non-injective encodings which outperforms the state of the art in terms of verification efficiency. Practical instantiations using these incomparable encoding schemes and the Poseidon2 arithmetization-oriented hash function are provided in [DKKW25b]. Notably, they all rely on an underlying 31-bit prime field making it a good fit with our MPC-friendly Poseidon2 instance over the KoalaBear prime field.

5.2 Previous Work on Threshold HBS

Threshold signature schemes enable distributed signing among a group of parties, where any subset of t -out-of- n participants can collaboratively generate a valid signature without reconstructing the private key. All previous attempts to turn HBS into threshold schemes are summarized below.

STARK-based threshold HBS. The idea behind STARK-based threshold signatures is for an aggregator to combine individual signatures over the same message into a zero-knowledge proof that serves as the actual threshold signature. More precisely, given a k -out-of- n setting, the aggregator can generate a proof attesting that it verified k distinct signatures over the same message and that signers are part of the quorum. Combining such a construction with HBS has been first explored by Khaburzeniya, Chalkias, Lewi and Malvai [KCLM22], using WOTS with 1-bit chunks (instantiated with the Rescue-Prime hash function) along with STARKs. A valuable feature of this approach is its non-interactiveness: the aggregator only needs to collect individual signatures in order to compute the proof, without any additional communication. On the other hand, it inflates the signature size by over an order of magnitude, making the resulting threshold signatures incompatible with standard signature formats.

Distributed HBS using Boolean shares. Distributed variants of stateful HBS, including XMSS, have been explored by Kelsey, Lang and Lucks in [KLL25] where they introduce n -out-of- n and k -out-of- n threshold signature schemes which rely on Boolean shares. For the n -out-of- n setting, a trusted dealer starts from an existing list of OTS secret keys and splits all hash chain intermediate values by means of $n + 1$ Boolean shares. More precisely, to split the x -th position in the chain of the i -th secret key component, the trusted dealer generates n random values $r_i^0, \dots, r_i^n$ which are meant to be respectively distributed to each party. An additional share, which will be part of a common reference string (CRS), is obtained by computing $r_i^{\text{crs}} = r_i^1 \oplus \dots \oplus r_i^n \oplus H^x(\text{sk}_i)$. Note that this process must be performed for each chain position of each secret key component within each OTS key. As a result, each party can sign non-interactively by simply providing its relevant Boolean shares, provided the aggregator can access the CRS shares to reconstruct the signature components. While participant shares can be derived pseudo-randomly from private seeds known only to the respective parties, the CRS shares must be stored in their complete form, leading to substantial memory requirements. For instance, the LMS instance LMS_SHA256_M32_H15 combined with the LMOTS instance LMS_SHA256_N32_W4 [MCF19] requires a 1.1GiB CRS as highlighted in the original publication [KLL25]. To turn their n -of- n scheme into a k -of- n threshold scheme, they propose to assign each OTS key to a single quorum among the $\binom{n}{k}$ k -member subset. A major downside of this approach is the increased state management complexity: signers must not only track which keys have already been used, but also determine which key to use for each specific quorum. Beyond this complexity, the approach increases the required Merkle tree height to support the same number of OTS across all possible coalitions, thereby increasing both signature size and CRS memory requirements.

MPC-based threshold HBS. Finally, another approach is to leverage a linear secret sharing scheme to split WOTS secret keys into shares distributed among participants. However, it requires to jointly compute all hash function calls in an MPC fashion. This can be very challenging in practice, especially in low latency scenarios, as highlighted by the work from Cozzo and Smart [CS19] which estimates around 85 minutes to compute a single threshold SPHINCS+ signature with SHA-3 as the underlying hash function, within a local area network. A possible workaround would be to precompute shares of the entire secret hash chains, enabling efficient non-interactive signing. Note that this approach would lead to memory requirements roughly equivalent to those of the CRS in the Boolean sharing scheme described above. Consequently, MPC-based approaches prove advantageous over this method for use-cases without trusted third party for signature aggregation, implying that all participants must store the CRS regardless. Indeed, it offers similar memory requirements while providing threshold functionality via the underlying sharing scheme, without the complexity of coalition state management or signature size increases. Furthermore, it does not require a trusted dealer, and keys can be generated in a distributed manner if desired, although performance may become a bottleneck for large Merkle tree heights. Accordingly, we believe this approach merits re-evaluation for HBS schemes with more favorable MPC characteristics, as is the case for **LeanSig** instantiated with **Poseidon2**. Although the performance reported in Table 2 exceeds prior estimates from [CS19] by orders of magnitude for hash calculations over MPC, it may still be insufficient for practical geographically distributed deployments. Nevertheless, we next introduce implementation optimizations that further improve performance.

5.3 Implementation Strategies

The deployment of hash-based signatures over MPC presents unique optimization opportunities that stem from the inherent parallelism in their structure. We identify two primary strategies that dramatically improve practical performance beyond naive implementations.

5.3.1 Exploiting Structural Parallelism

Hash-based signatures exhibit parallelism at multiple levels that can be leveraged to minimize communication rounds in MPC protocols. At the permutation level, S-box evaluations can be batched together within each round. Furthermore, the OTS construction provides additional parallelization opportunities as each signature requires v independent hash chains, computed from distinct secret key components. Since these chains share no dependencies, we can evaluate all chains in parallel, effectively amortizing the communication cost across the entire OTS. Concretely, instead of requiring $\text{depth}(\pi) \cdot (w - 1) \cdot v$ rounds for sequential evaluation, the parallel approach maintains the same $\text{depth}(\pi) \cdot (w - 1)$ round depth while processing v times more data per round. This parallelization extends further when generating multiple OTS at a time. By batching k OTS computations together, we reduce the amortized round complexity to $\text{depth}(\pi) \cdot (w - 1)/k$ per signature. In practice, k is limited only by available memory and preprocessing capacity. For example, with $\text{depth}(\pi) = 28$, $w = 4$ and $k = 128$, the effective round cost drops below one round per OTS, making the online phase as efficient as for threshold-friendly signature schemes.

5.3.2 Precomputation and Time-Memory Trade-offs

A distinctive feature of hash-based signatures is the ability to precompute all possible signature components independently of the message to be signed. While this requires computing some hash chains beyond their necessary length (as not all components will use the full chain), it eliminates the online phase entirely for signature generation. Specifically, for each OTS key i and internal component j , we precompute all intermediate values of

the corresponding hash chain:

$$\begin{aligned} \text{chain}[i][j][0] &= sk[i][j] \\ \text{chain}[i][j][x] &= H(\text{chain}[i][j][x-1]) \quad \forall x \in \{1, \dots, w-1\} \end{aligned}$$

Assuming this is done for every OTS with a Merkle tree of height h , this yields a precomputed table of size $2^h \cdot v \cdot w$ field elements. While it is equivalent to the CRS memory requirements for the method based on Boolean shares [KLL25], it has the advantage of supporting any threshold configuration thanks to the underlying secret sharing scheme. That way, the precomputation phase can be executed during system initialization or idle periods, transforming signature generation into a simple table lookup. When a signature request arrives for message m with encoding $(x_1, \dots, x_v)$, parties merely retrieve and output their shares of $\text{chain}[i][j][x_j - 1]$ for the appropriate key index i . The precomputation can be performed alongside the MPC protocol’s offline phase for generating Beaver triples. Both computations are message-independent and can be executed during periods of low system activity. The resulting online signature generation requires only:

1. Computing the message encoding (local operation)
2. Retrieving precomputed partial signature components (memory access)
3. Reconstructing the full signature

This approach enables flexible deployment strategies: systems with abundant storage can precompute entire Merkle trees, while memory-constrained deployments might maintain a sliding window of upcoming signatures.

5.4 Benchmarks

5.4.1 Experimental Setup

First, we evaluate the practical performance of threshold HBS for a single signature generation, without leveraging the above-described time-memory tradeoff. For that benchmark, we consider three distinct **LeanSig** instantiations. Two of them correspond to the *hashing-optimized* and *tradeoff* instances from the original **LeanSig** specification [DKKW25b], defined by $(v, w, T) = (64, 8, 375)$ and $(48, 10, 326)$, respectively. Additionally, we introduce a third MPC-friendly instance defined by $(v, w, T) = (68, 4, 114)$ ¹⁰ that features shorter hash chains and smaller target-sum, making it more MPC-friendly. We stress that all instances achieve the same security levels, namely 128 and 64 bits against classical and quantum adversaries, respectively. Then, for the precomputed non-(online-)interactive implementation strategy, we run another benchmark where multiple one-time signature (OTS) computations are batched in parallel to amortize communication costs. Due to practical limitations of the MP-SPDZ compiler which generates an excessive number of instructions for larger batch sizes, we restrict our evaluation to at most $n_\sigma = 4$ parallel signatures. Our benchmarks follow the same setup configuration as described in Section 4.

5.4.2 Results Interpretation

Online interactive signatures. Table 3 presents performance metrics for generating a single signature using three parameter sets. For the most MPC-friendly instance defined by $(v, w, T) = (68, 4, 114)$, both honest majority protocols complete hash chain computations under 0.5 and 2 seconds in the 1ms and 10ms latency settings, respectively, which is consistent with the results from Table 2. Another point worth mentioning that is not

¹⁰This instance was obtained using the script publicly available at <https://github.com/b-wagn/hypercube-hashsig-parameters>.

Table 3: Benchmarks of threshold signatures for multiple **LeanSig** instantiations defined by the tuple (v, w, T) , in a three-party setting. Only hash chains are considered (*i.e.*, calculations that do not require MPC are taken into account). Timing executions are reported for different single-trip delays (1ms, 10ms and 100ms) to reflect various network conditions.

(v, w, T)	Protocol	Prec.	Offline phase				Online phase				
			Time (s)			Data (MB)	Time (s)				Data (MB)
			$d = 1$	$d = 10$	$d = 100$		Com. rounds	$d = 1$	$d = 10$	$d = 100$	
(48, 10, 326)	MAMA	96 496	12	56	607	1 478	2 029	1.96	18	178	2.37
	MASCOT	96 496	5.76	24	228	694	2 029	1.94	18	178	2.36
	ATLAS	48 248	0.05	0.31	2.85	0.72	518	0.62	5.47	52	1.29
	Mal. Shamir	96 496	0.15	0.89	8.25	5.61	254	0.38	2.77	26	2.32
(64, 8, 375)	MAMA	111 000	14	64	694	1 691	1 581	1.57	14	139	2.71
	MASCOT	111 000	6.51	28	258	793	1 581	1.56	14	139	2.70
	ATLAS	55 000	0.05	0.36	3.34	0.84	408	0.53	4.28	41	1.49
	Mal. Shamir	111 000	0.18	1.04	9.34	6.48	198	0.33	2.16	20	2.66
(68, 4, 114)	MAMA	33 744	4.46	20	213	516	685	0.66	6.10	60	0.83
	MASCOT	33 744	2.03	8.71	82	241	685	0.66	6.10	60	0.83
	ATLAS	16 872	0.02	0.14	1.32	0.27	176	0.22	1.82	18	0.46
	Mal. Shamir	33 744	0.06	0.37	3.54	1.98	86	0.13	0.93	8.67	0.81

Table 4: Benchmarks for full precomputations of threshold hash-based signatures using the MP-SPDZ framework, in a three-party setting. The considered **LeanSig** instance is defined by the parameters $(v, w, T) = (68, 4, 114)$. n_σ indicates the number of OTS computed simultaneously. Only hash chains are considered (*i.e.*, calculations that do not require MPC are taken into account). Timing executions are reported for different single-trip delays (1ms, 10ms and 100ms) to reflect various network conditions.

n_σ	Protocol	Prec.	Offline phase				Com. rounds	Online phase				Data (MB)	Combined			
			Time (s)			Data (MB)		Time (s)			Data (MB)		Time (s)			Data (MB)
			$d = 1$	$d = 10$	$d = 100$			$d = 1$	$d = 10$	$d = 100$			$d = 1$	$d = 10$	$d = 100$	
1	MAMA	60 384	7.93	35	384	933	685	0.69	6.14	60	1.47	8.62	42	444	934	
	MASCOT	60 384	3.67	16	144	439	685	0.68	6.14	60	1.47	4.35	22	204	441	
	ATLAS	30 192	0.04	0.26	2.34	0.51	180	0.24	1.93	18	0.83	0.28	2.19	20	1.34	
	Mal. Shamir	60 384	0.11	0.72	6.44	3.54	86	0.16	0.97	8.83	1.45	0.27	1.69	15	4.99	
2	MAMA	120 768	16	70	756	1 842	685	0.77	6.22	60	2.92	16	76	816	1 845	
	MASCOT	120 768	7.09	30	279	864	685	0.74	6.20	60	2.92	7.84	37	339	867	
	ATLAS	60 384	0.06	0.43	3.86	0.95	186	0.30	2.06	19	1.64	0.36	2.49	23	2.59	
	Mal. Shamir	120 768	0.19	1.18	11	7.04	86	0.21	1.02	8.90	2.90	0.41	2.20	20	9.94	
4	MAMA	241 536	41	149	1 506	3 670	685	1.11	6.55	61	5.81	42	156	1 567	3 676	
	MASCOT	241 536	19	65	555	1 714	684	1.04	6.47	61	5.81	20	71	616	1 720	
	ATLAS	120 768	0.15	0.79	6.93	1.91	198	0.57	2.45	20	3.25	0.72	3.24	27	5.15	
	Mal. Shamir	241 536	0.52	2.33	20	15	86	0.42	1.30	9.24	5.80	0.94	3.63	29	21	

reflected in the benchmark results is the relevance of constant-sum encoding schemes for hash-based signatures over MPC. Such encodings are highly beneficial for MPC deployments as they allow participants to determine in advance the exact number of preprocessed elements (*e.g.*, Beaver triples) required for the online phase, enabling efficient resource allocation. However, it is less of a concern when leveraging the full precomputation strategy since all hash chains must be computed to their maximum length regardless of the actual message encoding. While the reported timing results constitute a considerable improvement over previous results for MPC-based threshold HBS, it may still fall short of low-latency requirements.

Precomputed non-(online-)interactive signatures. Table 4 reveals a different performance landscape when employing the full precomputation strategy where the offline phase scales linearly with the number of parallel signatures. Notably, the relative performance of protocols shifts as batch size increases. For $n_\sigma = 4$ with 10ms latency, ATLAS outperforms Malicious Shamir (3.24 vs 3.63 seconds combined time, respectively), despite the latter being faster at $n_\sigma = 1$ (2.19 vs 1.69 seconds combined time, respectively). This crossover point shows that protocols optimizing offline phase efficiency become increasingly advantageous as parallelism scales. The comparison between MAMA and MASCOT illustrates this clearly since their performance gap grows from approximately 2x to 3x as n_σ increases from 1 to 4. This suggests that applications requiring active security with dishonest majority and high statistical security should reconsider our initial conclusion in Section 3 regarding the most MPC-friendly Poseidon2 instances, and evaluate 64-bit prime fields instead. Indeed, larger prime fields would enable statistical security ≥ 40 bits without MAC duplication, a key drawback of small prime fields, thereby reducing offline phase computation costs. Overall, the fact that full precomputation completes in under one second per signature using ATLAS and Malicious Shamir in the 10ms latency setting with batch sizes $n_\sigma = 4$ is encouraging as better performance per signature can be expected as n_σ increases.

6 Conclusion and Future Work

This work re-evaluates the practicality of evaluating hash chains within MPC and demonstrates their viability for MPC-based threshold HBS. Whereas prior work concluded that such constructions were not deployable in practice, we show that, with AO-based primitives and targeted implementation optimizations, end-to-end signing via MPC-evaluated hash chains can be completed in under one second per signature on a wide-area network with ≈ 20 ms round-trip time. These results do not by themselves guarantee deployability across all environments, but will hopefully reopen the design space and refine the cost model for practitioners. While HBS are not the most efficient option for post-quantum threshold signatures overall, some systems may be constrained to hash-based primitives. An example are distributed validator solutions for Ethereum, which is considering a transition from BLS to hash-based signatures for its consensus layer. In such settings, practicality will depend critically on the chosen HBS instantiation and on system-level timing requirements (*e.g.*, slot durations) under the assumed network conditions. Our results provide concrete data points for evaluating those trade-offs. We hereafter underline interesting further research directions as well as several challenges that remain to be addressed.

LeanSig instantiations with minimized signature generation cost. The encoding scheme used by the considered LeanSig instantiations focuses on efficient signature verifications (*i.e.*, maximize the number of hash calls in the signature generation to minimize verification cost). Leveraging a variant of their scheme which minimizes the signing cost by decreasing the w and T parameters would lead to more MPC-friendly LeanSig instantiations. Indeed,

while the considered instances already employ short hash chains, having a smaller target-sum would allow to lighten the offline phase, which would be of high interest for the precomputed non-interactive implementation strategy.

Active security in dishonest-majority settings. Our benchmarks reveal that achieving active security against dishonest majorities remains computationally expensive, with MASCOT requiring orders of magnitude more preprocessing and communication than honest-majority protocols. Future work could explore specialized MPC protocols tailored to repeated, deterministic S-box evaluations (*e.g.*, amortized checks across many S-boxes) that could reduce the overhead of active security in the dishonest-majority setting. More specifically to Poseidon2, it would be valuable to investigate the performance benefits of leveraging larger prime fields (*e.g.*, 64-bit primes) for applications where 31-bit statistical security is not enough.

Robust threshold HBS. In this work, we assume threshold hash-based signatures by simply distributing the signing operation over MPC (*i.e.*, hash chain calculations). Indeed, while the complete signature is publicly verifiable through its Merkle tree, the partial signatures generated by individual participants lack a corresponding verification mechanism. This limitation could be exploited in scenarios where adversaries might contribute invalid partial signatures that only manifest as failures after the expensive MPC computation completes. Introducing a verifiability mechanism for partial signatures (*e.g.*, through auxiliary Merkle trees for threshold participants or zero-knowledge proofs of correct partial signature generation) could provide robustness without compromising the post-quantum security properties of the underlying hash-based construction.

Acknowledgements

We thank the anonymous reviewers for their insightful feedback and suggestions, as well as Marcel Keller for its support with the MP-SPDZ framework.

References

- [AABS⁺20] Abdelrahman Aly, Tomer Ashur, Eli Ben-Sasson, Siemen Dhooghe, and Alan Szepieniec. Design of Symmetric-Key Primitives for Advanced Cryptographic Protocols. *IACR Transactions on Symmetric Cryptology*, 2020(3):1–45, Sep. 2020.
- [BBLP22] Augustin Bariant, Clémence Bouvier, Gaëtan Leurent, and Léo Perrin. Algebraic Attacks against Some Arithmetization-Oriented Primitives. *IACR Transactions on Symmetric Cryptology*, 2022(3):73–101, Sep. 2022.
- [BC93] Jurjen N. E. Bos and David Chaum. Provably Unforgeable Signatures. In Ernest F. Brickell, editor, *Advances in Cryptology — CRYPTO’ 92*, pages 1–14, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.
- [BDH11] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. XMSS - A Practical Forward Secure Signature Scheme Based on Minimal Security Assumptions. In Bo-Yin Yang, editor, *Post-Quantum Cryptography*, pages 117–129, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [Bea92] Donald Beaver. Efficient Multiparty Protocols Using Circuit Randomization. In Joan Feigenbaum, editor, *Advances in Cryptology — CRYPTO ’91*, pages 420–432, Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.

- [BHK⁺19] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS+ Signature Framework. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS '19*, page 2129–2146, New York, NY, USA, 2019. Association for Computing Machinery.
- [CAD⁺20] David A. Cooper, Daniel C. Apon, Quynh H. Dang, Michael S. Davidson, Morris J. Dworkin, and Carl A. Miller. Recommendation for Stateful Hash-Based Signature Schemes, 2020-10-01 04:10:00 2020.
- [CS19] Daniele Cozzo and Nigel P. Smart. Sharing the LUOV: Threshold Post-quantum Signatures. In Martin Albrecht, editor, *Cryptography and Coding*, pages 128–153, Cham, 2019. Springer International Publishing.
- [DKKW25a] Justin Drake, Dmitry Khovratovich, Mikhail Kudinov, and Benedikt Wagner. Hash-Based Multi-Signatures for Post-Quantum Ethereum. *IACR Communications in Cryptology*, 2(1), 2025.
- [DKKW25b] Justin Drake, Dmitry Khovratovich, Mikhail Kudinov, and Benedikt Wagner. Technical Note: LeanSig for Post-Quantum Ethereum. Cryptology ePrint Archive, Paper 2025/1332, 2025.
- [DN07] Ivan Damgård and Jesper Buus Nielsen. Scalable and Unconditionally Secure Multiparty Computation. In Alfred Menezes, editor, *Advances in Cryptology - CRYPTO 2007*, pages 572–590, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
- [GKL⁺24] Lorenzo Grassi, Dmitry Khovratovich, Reinhard Lüftenegger, Christian Rechberger, Markus Schofnegger, and Roman Walch. Monolith: Circuit-Friendly Hash Functions with New Nonlinear Layers for Fast and Constant-Time Implementations. *IACR Transactions on Symmetric Cryptology*, 2024(3):44–83, Sep. 2024.
- [GKR⁺21] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 519–535. USENIX Association, August 2021.
- [GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A Faster Version of the Poseidon Hash Function. In *Progress in Cryptology - AFRICACRYPT 2023: 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19–21, 2023, Proceedings*, page 177–203, Berlin, Heidelberg, 2023. Springer-Verlag.
- [GLO⁺21] Vipul Goyal, Hanjun Li, Rafail Ostrovsky, Antigoni Polychroniadou, and Yifan Song. ATLAS: Efficient and Scalable MPC in the Honest Majority Setting. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021*, pages 244–274, Cham, 2021. Springer International Publishing.
- [GLR⁺20] Lorenzo Grassi, Reinhard Lüftenegger, Christian Rechberger, Dragos Rotaru, and Markus Schofnegger. On a Generalization of Substitution-Permutation Networks: The HADES Design Strategy. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Proceedings*, volume 12106 of *Lecture Notes in Computer Science*

- (including subseries *Lecture Notes in Artificial Intelligence* and *Lecture Notes in Bioinformatics*), pages 674–704. Springer, Cham, May 2020. Eurocrypt 2020 ; Conference date: 11-05-2020 Through 15-05-2020.
- [GLSV14] Vincent Grosso, Gaëtan Leurent, François-Xavier Standaert, and Kerem Varici. LS-Designs: Bitslice Encryption for Efficient Masked Software Implementations. In *FSE*, pages 18–37. Springer, 2014.
- [GØSW23] Lorenzo Grassi, Morten Øyegarden, Markus Schofnegger, and Roman Walch. From Farfalle to Megafono via Ciminion: The PRF Hydra for MPC Applications. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 255–286, Cham, 2023. Springer Nature Switzerland.
- [GRR98] Rosario Gennaro, Michael O. Rabin, and Tal Rabin. Simplified VSS and Fast-track Multiparty Computations with Applications to Threshold Cryptography. In *Proceedings of the Seventeenth Annual ACM Symposium on Principles of Distributed Computing*, PODC ’98, page 101–111, New York, NY, USA, 1998. Association for Computing Machinery.
- [GRR⁺16] Lorenzo Grassi, Christian Rechberger, Dragos Rotaru, Peter Scholl, and Nigel P. Smart. MPC-Friendly Symmetric Key Primitives. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’16, page 430–443, New York, NY, USA, 2016. Association for Computing Machinery.
- [Hal95] Neil M. Haller. The S/KEY One-Time Password System. RFC 1760, February 1995.
- [HBG⁺18] A. Huelsing, D. Butin, S. Gazdag, J. Rijneveld, and A. Mohaisen. XMSS: eXtended Merkle Signature Scheme. RFC 8391, RFC Editor, May 2018.
- [KCLM22] Irakli Khaburzaniya, Konstantinos Chalkias, Kevin Lewi, and Harjasleen Malvai. Aggregating and Thresholdizing Hash-based Signatures using STARKs. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, ASIA CCS ’22, page 393–407, New York, NY, USA, 2022. Association for Computing Machinery.
- [Kel20] Marcel Keller. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’20, page 1575–1590, New York, NY, USA, 2020. Association for Computing Machinery.
- [KKW25] Dmitry Khovratovich, Mikhail Kudinov, and Benedikt Wagner. At the Top of the Hypercube - Better Size-Time Tradeoffs for Hash-Based Signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 93–123, Cham, 2025. Springer Nature Switzerland.
- [KLL25] John Kelsey, Nathalie Lang, and Stefan Lucks. Turning Hash-Based Signatures into Distributed Signatures and Threshold Signatures. *IACR Communications in Cryptology*, 2(2), 2025.
- [KMB17] Dmitry Kogan, Nathan Manohar, and Dan Boneh. T/Key: Second-Factor Authentication From Secure Hash Chains. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’17, page 983–999, New York, NY, USA, 2017. Association for Computing Machinery.

- [KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: Faster Malicious Arithmetic Secure Computation with Oblivious Transfer. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 830–842, New York, NY, USA, 2016. Association for Computing Machinery.
- [Lam79] Leslie Lamport. Constructing Digital Signatures from a One Way Function. Technical Report CSL-98, SRI International, October 1979.
- [Lam81] Leslie Lamport. Password Authentication With Insecure Communication. *Commun. ACM*, 24(11):770–772, November 1981.
- [LM95] Thomson Leighton and Silvio Micali. Large provably fast and secure digital signature schemes based on secure hash functions, U.S. Patent 5,432,852, July 1995.
- [LM24] Charlotte Lefevre and Bart Mennink. Permutation-Based Hash Chains with Application to Password Hashing. *IACR Transactions on Symmetric Cryptology*, 2024(4):249–286, Dec. 2024.
- [LN17] Yehuda Lindell and Ariel Nof. A Framework for Constructing Fast MPC over Arithmetic Circuits with Malicious Adversaries and an Honest-Majority. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, page 259–276, New York, NY, USA, 2017. Association for Computing Machinery.
- [MCF19] David McGrew, Michael Curcio, and Scott Fluhrer. Leighton-Micali Hash-Based Signatures. RFC 8554, April 2019.
- [Mer79] Ralph Charles Merkle. *Secrecy, authentication, and public key systems*. PhD thesis, Stanford University, Stanford, CA, USA, 1979. AAI8001972.
- [Rab78] Michael O. Rabin. Digitalized Signatures. In *Foundations of Secure Computation*, page 155–168, December 1978.
- [Vau93] S. Vaudenay. One-Time Identification with Low Memory. In P. Camion, P. Charpin, and S. Harari, editors, *Eurocode '92: International Symposium on Coding Theory and Applications*, pages 217–228, Vienna, 1993. Springer Vienna.
- [YM24] Burcu Yıldız and Mary Maller. Comparing Two Hash Functions for Multi-Party Computation and Zero-Knowledge , September 2024.